

AMLGuard

Manual de Estudo - Volume 3

CI/CD com GitHub Actions e publicacao de imagem no GHCR

Fase	Dias	Resultado
Automacao e distribuicao	11 e 12	Testes automatizados, lint, build e publicacao da imagem Docker.

Objetivo do volume: compreender o que mudou quando o AMLGuard deixou de depender apenas de verificacoes locais e passou a possuir um processo remoto, repetivel e auditavel para validar o codigo e distribuir sua imagem Docker.

Evidencias reais desta fase

- 44 testes aprovados em ambiente local e no GitHub Actions.
- Ruff limpo para o codigo Python do produto e dos testes.
- Hadolint limpo para o Dockerfile.
- Imagem Docker publicada automaticamente no GitHub Container Registry.
- Dependencia entre jobs: a publicacao so ocorre depois da aprovacao dos controles de qualidade.

Mensagem central: CI/CD nao e apenas uma ferramenta. E uma regra de processo: nenhum artefato deve ser distribuido sem passar por verificacoes reproduzeis.

1. O que e integracao continua

Analogia da oficina: imagine uma linha de montagem em que cada peça nova precisa ser testada antes de entrar no carro. O GitHub Actions funciona como uma estação automática de inspeção. Sempre que um commit chega, a estação monta o ambiente, instala as dependências, executa os testes e verifica a qualidade do código.

No AMLGuard, o fluxo do Dia 11 ficou assim:

Evento	Job	Pergunta respondida
Push ou pull request	python-checks	O código instala, passa no Ruff e os 44 testes continuam verdes?
Push ou pull request	dockerfile-lint	O Dockerfile segue boas práticas e não contém erros estruturais?

Conceitos essenciais

Workflow: Arquivo YAML que descreve quando e como a automação deve executar.

Job: Conjunto de passos que roda em uma máquina virtual isolada.

Step: Uma ação individual, como instalar dependências ou executar pytest.

Runner: Computador temporário fornecido pelo GitHub para executar o job.

Trigger: Evento que inicia o workflow, como push ou pull request.

Concurrency: Regra que cancela execuções antigas quando uma nova substitui a anterior.

Autoteste 1

1. Qual a diferença entre workflow, job e step?
2. Por que executar os testes em uma máquina do GitHub é mais confiável do que apenas no computador do desenvolvedor?
3. O que a regra de concurrency evita?

Respostas: o workflow é o processo completo; o job é uma unidade de execução; o step é uma ação dentro do job. O runner remoto reduz dependência de configurações locais. A concurrency evita gastar recursos com execuções antigas que já foram superadas.

2. De CI para CD

Analogia da fabrica e do centro de distribuicao: no Dia 11, a fabrica apenas testava o produto. No Dia 12, depois da aprovacao, o produto passou a ser embalado e enviado ao estoque central. O produto e a imagem Docker; o estoque e o GHCR.

Neste projeto, CD significa continuous delivery: a imagem e criada e publicada automaticamente. Isso ainda nao significa que ela foi implantada em um servidor de producao. Publicar e implantar sao etapas diferentes.

Fluxo implementado

Etapa	Passo	Funcao
1	Checkout	Baixa o commit exato que disparou a execucao.
2	Buildx	Prepara o mecanismo moderno de build do Docker.
3	Login GHCR	Autentica com o token temporario do GitHub.
4	Metadata	Gera tags e labels de rastreabilidade.
5	Build and push	Constroi a imagem e envia ao registro.
6	Summary	Registra imagem e digest no resumo da execucao.

Por que o job publish-image usa needs

A dependencia needs transforma os testes em um portao. Sem ela, o build poderia publicar uma imagem mesmo que pytest, Ruff ou Hadolint falhassem. Com ela, a distribuicao so ocorre depois da aprovacao dos dois jobs anteriores.

Autoteste 2

1. Publicar uma imagem no GHCR significa que ela ja esta em producao?
2. Por que publish-image nao deve rodar em qualquer pull request?
3. O que aconteceria se o job nao dependesse dos testes?

Respostas: nao; a imagem esta armazenada, mas nao implantada. Pull requests ainda podem ser alterados e nao representam a versao oficial da main. Sem dependencia, um artefato defeituoso poderia ser publicado.

3. Tags, digest e rastreabilidade

Uma imagem pode ter varios nomes apontando para o mesmo conteudo. Esses nomes sao as tags. O digest, por outro lado, identifica criptograficamente o conteudo exato da imagem.

Referencia	Comportamento	Uso recomendado
latest	Movel: aponta para a publicacao mais recente.	Demonstracao e uso humano simples.
main	Movel: acompanha a branch principal.	Ambientes de desenvolvimento ou homologacao.
sha-...	Imutavel na pratica: corresponde a um commit especifico.	Reproducao, auditoria e rollback.
digest	Identifica os bytes exatos da imagem.	Implantacoes que exigem maxima garantia de integridade.

GITHUB_TOKEN

O workflow nao armazena uma senha pessoal. Durante a execucao, o GitHub cria um token temporario com as permissoes declaradas. O job recebeu contents: read para ler o repositorio e packages: write para publicar no GHCR.

Principio de menor privilegio: uma automacao deve receber somente as permissoes necessarias. Menos permissoes significam menor impacto em caso de erro ou comprometimento.

Autoteste 3

1. Por que latest nao e suficiente para auditoria?
2. Qual referencia permite reconstruir a relacao entre imagem e commit?
3. Por que nao devemos colocar token pessoal diretamente no YAML?

Respostas: latest muda com o tempo. A tag baseada em SHA relaciona a imagem ao commit e o digest garante o conteudo exato. Segredos no YAML ficam expostos no historico do Git; o token temporario evita esse risco.

4. Licoes e erros reais do AMLGuard

Patch onde o simbolo e usado

No Dia 8, o monkeypatch precisou atingir `src.api.main.get_model` e `src.models.predict.get_model`. Imports copiam referencias; testes devem patchar o ponto de uso.

Escopo correto do Ruff

No Dia 9, verificar o repositório inteiro incluiu notebook e scripts históricos fora do escopo. A verificação foi ajustada para o código do produto e os testes.

Disk usage não e image size

No Dia 10, 2,15 GB de uso local incluía cache e camadas intermediárias. O tamanho relevante da imagem publicada era o content size.

CI local não substitui CI remoto

Os 44 testes locais eram necessários, mas o GitHub Actions provou que o projeto funciona em ambiente limpo e repetível.

Publicar somente depois da qualidade

No Dia 12, `publish-image` foi condicionado aos jobs de teste e lint, transformando qualidade em regra de processo.

O que isso comunica a um recrutador

Evidencia	Sinal profissional
Workflow em push e PR	Compreensão de integração contínua.
Jobs paralelos	Uso eficiente de automação e isolamento de responsabilidades.
Gate antes da publicação	Mentalidade de qualidade e controle de release.
Tags e labels OCI	Rastreabilidade e governança de artefatos.
GHCR	Capacidade de distribuir uma aplicação containerizada.

5. Revisao final da fase

Explique em voz alta, sem consultar:

- [] O que inicia um workflow do GitHub Actions.
- [] A diferenca entre workflow, job, step e runner.
- [] Por que pytest, Ruff e Hadolint possuem responsabilidades diferentes.
- [] Como needs impede a publicacao de uma imagem reprovada.
- [] A diferenca entre CI, continuous delivery e deployment.
- [] A funcao do GHCR e a diferenca entre latest, SHA e digest.
- [] Por que GITHUB_TOKEN e preferivel a um token escrito no repositorio.

Desafio de entrevista

Pergunta: Como voce garantiria que uma imagem Docker defeituosa nao fosse publicada?

Resposta esperada: Eu separaria verificacoes em jobs independentes, como testes Python e lint do Dockerfile. O job de build e push dependeria explicitamente desses jobs e executaria somente em push para a branch principal. Assim, qualquer falha interromperia a cadeia antes da publicacao. Eu tambem usaria tags por commit e registraria o digest para rastreabilidade.

Checklist da fase

- [x] 44 testes automatizados aprovados.
- [x] Ruff aprovado.
- [x] Hadolint aprovado.
- [x] Workflow executado em push para main.
- [x] Publicacao condicionada aos jobs de qualidade.
- [x] Imagem enviada ao GHCR.
- [x] Evidencia visual da execucao verde.

Proxima fase: rastreamento de experimentos com MLflow. O objetivo sera transformar cada treinamento em um registro comparavel, contendo parametros, metricas, artefatos e identificacao da versao executada.